



LAPORAN PRAKTIKUM 4 - STATEMENT PENENTU KEPUTUSAN

Nama : Muhammad Fadil Adiatma

NRP : 3226600139

Kelas : 1 TK C

Dosen Pengampu: Dr. Ir. Sigit Wasista M. Kom

PERCOBAAN 4

STATEMENT PENENTU KEPUTUSAN

4.1 Tujuan Praktikum

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Menjelaskan konsep statement penentu keputusan dalam program.
- Menggunakan statement **if** untuk menjalankan proses berdasarkan suatu kondisi.
- Menggunakan **if-else** untuk memilih salah satu dari dua kemungkinan proses.
- Menggunakan **else-if** untuk memilih satu kondisi dari beberapa kondisi yang tersedia.
- Menggunakan compound statement untuk mengelompokkan beberapa statement dalam satu blok.
- Menggunakan **switch-case** sebagai alternatif pemilihan berdasarkan nilai tertentu.
- Menggunakan operator relasional dan operator logika dalam pembentukan kondisi.
- Menguji program dengan beberapa nilai masukan dan menjelaskan hasil eksekusinya.

4.2 Dasar Teori

Statement penentu keputusan digunakan untuk mengatur alur eksekusi program berdasarkan kondisi tertentu. Kondisi dievaluasi sebagai benar (true) atau salah (false). Jika kondisi terpenuhi, program menjalankan statement yang sesuai; jika tidak terpenuhi, program dapat menjalankan statement alternatif.

Kondisi pada statement penentu keputusan umumnya dibentuk menggunakan operator relasional dan/atau operator logika. Beberapa bentuk yang digunakan pada praktikum ini adalah if, if-else, else-if, compound statement, nested if, dan switch-case.

4.2.1 Operator Relasional

Operator	Makna	Contoh
==	sama dengan	a == b
!=	tidak sama dengan	a != b
>	lebih besar	a > b
<	lebih kecil	a < b
>=	lebih besar atau sama dengan	a >= b
<=	lebih kecil atau sama dengan	a <= b

4.2.2 Operator Logika

Operator	Makna	Contoh
&&	AND; semua kondisi harus benar	a > 0 && a < 10
 	OR; minimal satu kondisi benar	a < 0 a > 100
!	NOT; membalik nilai logika	!(a == 0)

4.2.3 Statement if

Statement if digunakan ketika suatu proses hanya perlu dijalankan apabila kondisi tertentu terpenuhi.

```
if (kondisi) {  
    statement;  
}
```

4.2.4 Statement if-else

Statement if-else digunakan ketika terdapat dua kemungkinan proses: satu proses dijalankan jika kondisi benar dan proses lainnya jika kondisi salah.

```
if (kondisi) {  
    statement_jika_benar;  
} else {  
    statement_jika_salah;  
}
```

4.2.5 Statement else-if

Statement else-if digunakan ketika terdapat beberapa kondisi yang perlu diperiksa secara berurutan. Program menjalankan blok pertama yang kondisinya terpenuhi.

```
if (kondisi_1) {  
    statement_1; } else if  
(kondisi_2) {  
    statement_2; } else {  
    statement_lain;  
}
```

4.2.6 Nested if

Nested if adalah penggunaan statement if di dalam statement if lainnya. Bentuk ini dapat digunakan ketika keputusan kedua baru relevan setelah keputusan pertama terpenuhi.

```
if (kondisi_1) {  
    if (kondisi_2) {  
        statement;  
    }  
}
```

4.2.7 Compound Statement

Compound statement adalah beberapa statement yang dikelompokkan dalam satu blok menggunakan tanda kurung kurawal { }. Pengelompokan ini penting agar beberapa proses diperlakukan sebagai satu bagian dari if, else, atau struktur keputusan lainnya.

4.2.8 Statement switch-case

Statement switch-case digunakan untuk memilih proses berdasarkan nilai suatu ekspresi. Setiap nilai pilihan ditulis dalam case. Statement break digunakan untuk menghentikan eksekusi pada case yang sesuai, sedangkan default dijalankan apabila tidak ada case yang cocok.

Bagian	Fungsi
switch	Menentukan ekspresi yang diperiksa.
case	Menentukan nilai pilihan.
break	Menghentikan eksekusi switch.
default	Menangani kondisi ketika tidak ada case yang sesuai.
Catatan: Pada modul ini digunakan sintaks C yang lebih umum, yaitu <code>int main()</code> , <code>#include <stdio.h></code> , dan <code>return 0</code> . Fungsi <code>getch()</code> dari <code>conio.h</code> tidak digunakan karena bukan bagian dari pustaka standar C.	

4.3 Alat dan Bahan

- Komputer/laptop.
- Compiler atau IDE yang mendukung bahasa C.
- Editor kode sumber.
- Modul praktikum.

4.4 Program Percobaan

4.4.1 Percobaan 1 – Statement if

Program berikut menentukan nilai absolut suatu bilangan. Jika bilangan bernilai negatif, tandanya diubah menjadi positif. Jika tidak negatif, nilainya tetap.

```
#include <stdio.h>
int
main(void)
{
    int bil,
    absolut;
    printf("Masukkan bilangan bulat:
");
    scanf("%d", &bil);
    absolut = bil;
    if (bil < 0)
        absolut = -bil;
    printf("Nilai absolut dari %d adalah %d\n", bil,
absolut);
    return
0;
}
```

Hasil Running

```
Masukkan bilangan bulat: -2
Nilai absolut dari -2 adalah 2
```

Analisis program:

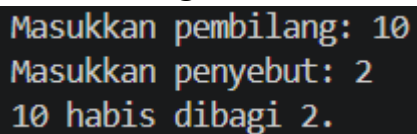
- **Input:** Saya memasukkan angka -2
- **Kondisi:** $bil < 0$.
- **Proses:** Jika kondisi benar, absolut diisi -bil.
- **Output:** Nilai absolut dari -2 adalah 2
- **Konsep:** Karena input menunjukkan angka < 0 maka program akan mengoutput hasil absolut dari angka tersebut dengan merubahnya ke positif menggunakan formula “-bil”.

4.4.2 Percobaan 2 – if-else dan Operator Modulus

Operator modulus (%) menghasilkan sisa pembagian bilangan bulat. Program menggunakan sisa pembagian untuk menentukan apakah pembagian habis atau tidak.

```
#include <stdio.h>
int
main(void)
{
    int pembilang, penyebut,
    sisa;
    printf("Masukkan pembilang:");
    scanf("%d", &pembilang);
    printf("Masukkan penyebut:");
    scanf("%d", &penyebut);
    if (penyebut == 0) {
        printf("Penyebut tidak boleh 0.\n");
    } else {
        sisa =
        pembilang % penyebut;
        if (sisa != 0)
            printf("%d tidak habis dibagi
            %d.\n", pembilang, penyebut);
        else
            printf("%d habis
            dibagi %d.\n", pembilang, penyebut);
    }
    return 0;
}
```

Hasil Running



```
Masukkan pembilang: 10
Masukkan penyebut: 2
10 habis dibagi 2.
```

Analisis program:

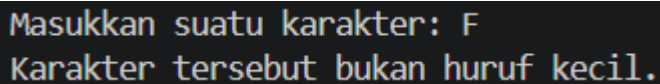
Struktur if-else digunakan untuk membuat keputusan berdasarkan suatu kondisi (nilai boolean: 1 untuk *true* atau 0 untuk *false*). Jika kondisi pada if terpenuhi, blok kode di dalamnya akan dijalankan; jika tidak, blok else yang akan dieksekusi. nested if adalah struktur if yang berada di dalam blok if atau else lainnya. Struktur ini digunakan ketika suatu keputusan bergantung pada beberapa kondisi bertingkat. Operator % digunakan untuk menghitung sisa bilangan bulat (*integer*). Dalam matematika dan pemrograman, pembagian dengan angka nol (0) tidak terdefinisi. Di bahasa C, membagi bilangan dengan nol saat program berjalan akan menyebabkan *runtime error*.

4.4.3 Percobaan 3 – Kondisi pada Karakter

Program berikut memeriksa apakah karakter yang dimasukkan termasuk huruf kecil a sampai z. Pemeriksaan dilakukan dengan operator relasional dan logika AND (&&).

```
#include <stdio.h>
int
main(void) {
    char c;
    printf("Masukkan suatu karakter:
");    scanf(" %c", &c);
    if (c >= 'a' && c <= 'z')        printf("Karakter
tersebut termasuk huruf kecil.\n");    else
printf("Karakter tersebut bukan huruf kecil.\n");
    return
0;
}
```

Hasil Running



```
Masukkan suatu karakter: F
Karakter tersebut bukan huruf kecil.
```

Analisis program:

Operator && adalah operator logika AND. Operator ini digunakan untuk menggabungkan dua atau lebih kondisi (ekspresi boolean). Kondisi `c >= 'a' && c <= 'z'` digunakan untuk memeriksa apakah suatu karakter yang tersimpan di variabel `c` merupakan huruf kecil (a–z).

4.4.4 Percobaan 4 – Demonstrasi goto

Percobaan ini mempertahankan konsep pada modul awal, yaitu kombinasi if dan goto untuk mengulang proses. Contoh menghitung kelipatan 2 secara berulang sampai iterasi ke-10.

```
#include <stdio.h>
int
main(void)
{
    int i = 0, hasil =
1;
    ulang:
    i++;    hasil
*= 2;
    printf("Iterasi ke-%2d: %5d\n", i,
hasil);
    if (i < 10)
goto ulang;

    return 0;
}
```

Hasil Running

```
Iterasi ke- 1:    2
Iterasi ke- 2:    4
Iterasi ke- 3:    8
Iterasi ke- 4:   16
Iterasi ke- 5:   32
Iterasi ke- 6:   64
Iterasi ke- 7:  128
Iterasi ke- 8:  256
Iterasi ke- 9:  512
Iterasi ke-10: 1024
```

Analisis program:

Label ulang: bertindak sebagai penanda lokasi (posisi) di dalam kode program. Label ini didefinisikan dengan menulis nama label yang diakhiri tanda titik dua (:). Perintah goto digunakan untuk melakukan percabangan tak bersyarat (unconditional jump). Kondisi if ($i < 10$) berfungsi sebagai penentu batas (kondisi henti) perulangan. Program ini mensimulasikan perulangan (*looping*) manual yang menghitung perpangkatan 2 (2^1 hingga 2^{10}) sebanyak 10 kali

Catatan: goto ditampilkan karena merupakan bagian dari materi sumber. Dalam pengembangan program, struktur perulangan for, while, atau do-while umumnya lebih mudah dibaca dan dipelihara untuk kasus perulangan seperti ini.

4.4.5 Percobaan 5 – Compound Statement

Compound statement digunakan ketika satu kondisi harus menjalankan beberapa statement sekaligus. Contoh berikut menentukan pilihan merek berdasarkan tipe kendaraan.

```
#include <stdio.h>
int
main(void)
{   char
    tag;

    printf("Masukkan tipe tag (2 / 4):
");   scanf(" %c", &tag);
    if (tag == '2') {           printf("Merek sepeda
motor bertipe 2 tag:\n");      printf("1.
YAMAHA\n");                   printf("2. SUZUKI\n");
printf("3. HONDA\n");
    } else {                   printf("Merek sepeda motor
bertipe 4 tag:\n");           printf("1. HONDA\n");
printf("2. SUZUKI\n");         printf("3. YAMAHA\n");
    }
    return 0;
}
```

Hasil Running

```
Masukkan tipe tag (2 / 4): 2
Merek sepeda motor bertipe 2 tag:
1. YAMAHA
2. SUZUKI
3. HONDA
```

Analisis program:

Beberapa statement printf() perlu dikelompokkan menggunakan tanda kurung kurawal { } karena tanda tersebut membentuk sebuah blok kode (compound statement).

4.4.6 Percobaan 6 – else-if

Program berikut memilih operasi aritmatika berdasarkan operator yang dimasukkan pengguna.

Struktur else-if digunakan untuk memeriksa beberapa kemungkinan secara berurutan.

```
#include <stdio.h>
int
main(void)
{
    char operator;
    float bil1, bil2, hasil;
    printf("Masukkan:          bilangan1          operator\n");
    printf("Operator: +   -   *   /\n");
    printf("Input: ");
    scanf("%f %c %f", &bil1, &operator,
    &bil2);
    if (operator == '*')          hasil =
bil1 * bil2;          else if (operator == '/')
hasil = bil1 / bil2;          else if (operator ==
'+')          hasil = bil1 + bil2;          else if
(operator == '-')          hasil = bil1 -
bil2;          else {          printf("Operator
tidak dikenali.\n");          return 0;
    }          printf("Hasil: %.2f %c %.2f =
%.2f\n",          bil1, operator, bil2,
hasil);
    return
0;
}
```

Hasil Running

```
Operator: +   -   *   /
Input: 2 + 3
Hasil: 2.00 + 3.00 = 5.00
```

Analisis program:

Program mengambil tiga input sekaligus menggunakan fungsi scanf: %f pertama: Membaca bilangan desimal (*float*) pertama, %c: Membaca satu karakter operator (+, -, *, atau /), %f kedua: Membaca bilangan desimal kedua dan menyimpannya di variabel bil2, Program memeriksa karakter pada variabel operator secara berurutan dari atas ke bawah, else if digunakan untuk membuat percabangan bersyarat majemuk (*multi-way branching*).

Catatan: Untuk penyempurnaan program, tambahkan pemeriksaan bil2 == 0 sebelum melakukan pembagian.

4.4.7 Percobaan 7 – switch-case

Program berikut merupakan alternatif dari percobaan sebelumnya. Pemilihan operasi dilakukan menggunakan switch-case.

```
#include <stdio.h>
int
main(void)
{
    char
    operator;
    float bil1, bil2, hasil;
    printf("Masukkan:          bilangan1          operator\n");
    printf("bilangan2\n");      printf("Operator: +   -   *   /\n");
    printf("Input: ");
    scanf("%f %c %f", &bil1, &operator,
    &bil2);
    switch (operator) {
    case '*':          hasil =
    bil1 * bil2;          break;
    case '/':
        if (bil2 == 0) {          printf("Pembagian dengan
    nol tidak diperbolehkan.\n");          return 0;
        }          hasil
    = bil1 / bil2;
    break;          case '+':
    hasil = bil1 + bil2;
    break;          case '-':
    hasil = bil1 - bil2;
    break;          default:
        printf("Operator tidak dikenali.\n");
    return 0;
    }
    printf("Hasil: %.2f %c %.2f =
    %.2f\n",          bil1, operator, bil2,
    hasil);
    return
    0;
}
```

Analisis program:

switch adalah struktur kontrol keputusan (*control flow*) yang digunakan untuk mengevaluasi satu nilai ekspresi/variabel, lalu mengarahkan eksekusi program ke salah satu dari beberapa cabang yang cocok. case menandai titik masuk (*entry point*) atau label nilai yang dicocokkan dengan ekspresi pada switch. break adalah perintah untuk menghentikan eksekusi blok switch dan keluar dari struktur tersebut. default adalah cabang cadangan yang dieksekusi jika tidak ada satupun label case yang cocok dengan nilai ekspresi switch. Di dalam case '/', terdapat blok pengujian kondisi khusus sebelum operasi pembagian dilakukan, dalam matematika dan komputer, membagi angka dengan nol adalah operasi yang tidak terdefinisi.

4.5 Pengujian Program

Setiap program harus diuji dengan beberapa data masukan. Pengujian tidak hanya menggunakan data normal, tetapi juga kondisi batas dan data yang berpotensi menghasilkan kesalahan.

No.	Data Uji	Hasil yang Diharapkan	Hasil Aktual
1			
2			
3			
4			
5			

4.6 Analisis Hasil Praktikum

Jawablah pertanyaan berikut berdasarkan program yang telah dijalankan.

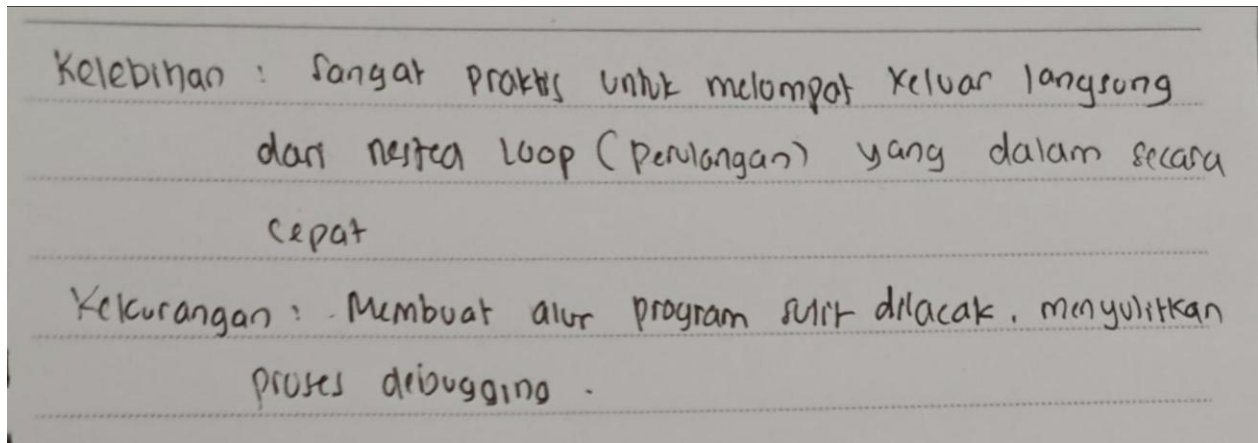
1. Apa perbedaan utama antara if dan if-else?
Jawaban: if hanya mengeksekusi blok kode jika kondisi bernilai *true* (dan tidak melakukan apa-apa jika *false*), sedangkan if-else menyediakan blok alternatif yang akan dieksekusi jika kondisi bernilai *false*.
2. Kapan struktur else-if lebih sesuai digunakan daripada beberapa if yang berdiri sendiri?
Jawaban: Saat ada beberapa kondisi yang saling menolak (*mutually exclusive*), di mana hanya boleh ada satu kondisi yang dieksekusi, sehingga program tidak perlu memeriksa kondisi lain jika salah satu sudah terpenuhi.
3. Mengapa operator && pada percobaan karakter menyebabkan kedua kondisi harus terpenuhi?
Jawaban: Karena && adalah operator logika AND, yang secara aturan hanya akan menghasilkan nilai *true* jika semua (kedua) kondisi di kiri dan kanannya bernilai *true*.
4. Apa fungsi tanda kurung kurawal { } pada compound statement?
Jawaban: Mengelompokkan beberapa pernyataan (*statement*) menjadi satu kesatuan blok kode yang akan dieksekusi bersamaan oleh struktur kontrol (seperti if, loop, atau fungsi).
5. Apa fungsi break pada switch-case?
Jawaban: Menghentikan eksekusi struktur switch dan keluar dari blok tersebut setelah suatu case yang cocok selesai dijalankan.
6. Apa yang terjadi jika break dihilangkan dari sebuah case?
Jawaban: Program akan terus mengeksekusi kode pada case berikutnya di bawahnya secara berurutan, tanpa peduli apakah kondisinya cocok atau tidak.

7. Mengapa pembagian dengan nol perlu ditangani secara khusus?

Jawaban: Karena pembagian dengan nol adalah operasi matematika yang tidak terdefinisi, yang dapat menyebabkan error *runtime*, program *crash*, atau menghasilkan nilai *undefined* (inf/NaN).

8. Apa kelebihan dan kekurangan penggunaan goto pada contoh percobaan?

Jawaban:



4.7 Tugas Praktikum

Tugas dikerjakan secara mandiri. Setiap program harus disertai kode sumber, minimal tiga data uji, hasil eksekusi, dan penjelasan singkat mengenai logika percabangannya.

4.7.1 Bilangan Ganjil atau Genap

Analisis Permasalahan

Program menerima input sebuah bilangan bulat dari pengguna. Masalah ini diselesaikan dengan memeriksa sisa hasil bagi (*modulus*) bilangan tersebut saat dibagi 2. Jika sisa hasil bagi adalah 0, maka bilangan tersebut genap. Jika sisa hasil bagi adalah 1 (atau tidak sama dengan 0), maka bilangan tersebut ganjil.

Algoritma//Logika Program

Program dimulai dengan menginisialisasi variabel integer bilangan untuk menampung input pengguna. Pengguna diminta memasukkan sebuah bilangan bulat, lalu program menghitung sisa hasil bagi (*modulus*) dari bilangan tersebut saat dibagi 2 menggunakan operator %. Program kemudian mengevaluasi kondisi: jika sisa hasil bagi sama dengan 0, maka program akan mencetak pernyataan bahwa bilangan tersebut adalah bilangan genap. Sebaliknya, jika sisa hasil bagi tidak sama dengan 0, program akan mencetak bahwa bilangan tersebut adalah bilangan ganjil, lalu eksekusi program berakhir.

Kode Program

```
1 #include <stdio.h>
2 int main ()
3 {
4     int angka;
5     char karakter;
6
7     printf("masukkan bilangan =");
8     scanf("%d", &angka);
9     if (angka % 2 == 0){
10         printf("bilangan anda (%d) adalah bilangan genap\n\n",angka);
11     } else {
12         printf("bilangan anda (%d) adalah bilangan ganjil\n\n",angka);
13     }
14 }
```

Hasil Running

```
masukkan bilangan =9
bilangan anda (9) adalah bilangan ganjil
```

Pengujian data

No	Input	Hasil yang diharapkan	Hasil Aktual	Keterangan
1	14	Genap	Bilangan anda (14) adalah bilangan genap	Valid
2	21	Ganjil	Bilangan anda (21) adalah bilangan ganjil	Valid
3	0	Genap	Bilangan anda (0) adalah bilangan genap	Valid

Analisis Hasil

Operator % (*modulus*) bekerja secara efektif untuk mengecek kelipatan 2. Pengujian terhadap angka positif ganjil, positif genap, maupun angka 0 menghasilkan luaran yang akurat.

4.7.2 Klasifikasi Karakter

Analisis Permasalahan

Input berupa satu karakter perlu dikategorikan berdasarkan rentang nilai ASCII atau menggunakan fungsi penguji karakter standar C. Kategori yang ditentukan mencakup huruf besar (A-Z), huruf kecil (a-z), digit angka (0-9), dan karakter khusus/symbol.

Algoritma/Logika Program

Program dimulai dengan menginisialisasi variabel bertipe karakter (*char*) dan meminta satu karakter input dari pengguna. Karakter tersebut dievaluasi secara sekuensial menggunakan struktur *else-if* berdasarkan nilai ASCII-nya. Pertama, program memeriksa apakah karakter berada dalam rentang 'A' hingga 'Z'. Jika ya, karakter diklasifikasikan sebagai huruf kapital. Jika tidak, program mengecek apakah karakter berada dalam rentang 'a' hingga 'z' untuk huruf kecil,

Kode Program

```
1  #include <stdio.h>
2  int main ()
3  {
4      char karakter;
5      printf("masukkan satu karakter =");
6      scanf("%c",&karakter);
7      printf("hasil karakter anda (%c) termasuk dalam kelompok ", karakter);
8
9      if(karakter >= 'a' && karakter <= 'z'){
10         printf("huruf kecil\n");
11     } else if (karakter >= 'A' && karakter <= 'Z'){
12         printf("Huruf besar\n");
13     } else if (karakter >= '0' && karakter <= '9'){
14         printf("angka");
15     } else {
16         printf("karakter khusus");
17     }
18 }
```

Hasil Running

```
masukkan satu karakter =A
hasil karakter anda (A) termasuk dalam kelompok Huruf besar
```

Pengujian Data

No	Input	Hasil yang diharapkan	Hasil aktual	Keterangan
1	A	Huruf besar	Hasil karakter anda (A) termasuk dalam kelompok Huruf besar	Valid
2	8	Angka	hasil karakter anda (8) termasuk dalam kelompok angka	Valid
3	@	Karakter khusus	hasil karakter anda (@) termasuk dalam kelompok karakter khusus	Valid

Analisis Hasil

Penggunaan operator logika && memastikan bahwa karakter berada persis dalam rentang yang ditentukan. Evaluasi secara sekuensial dengan else-if mencegah terjadinya klasifikasi ganda.

4.7.3 Kalkulator Sederhana

Analisis Permasalahan

Program menerima input dua bilangan riil (*float*) dan satu operator aritmatika (+, -, *, /). Program mengeksekusi perhitungan sesuai operator yang dipilih. Khusus untuk operasi pembagian, terdapat penanganan validasi agar divisor/pembagi tidak bernilai nol.

Algoritma/ Logika Program

Program diawali dengan menginisialisasi variabel accumulator bernilai 0 dan menjalankan perulangan tak terbatas (*while* (1)). Pada setiap iterasi, program meminta input dari pengguna berupa sebuah nilai bulat (bilangan) dan sebuah karakter (operator). Jika pembacaan input gagal (bukan 2 item yang valid), perulangan dihentikan dengan perintah *break*. Jika input valid, nilai operator dievaluasi menggunakan *switch-case*. Operator 's'/'S' digunakan untuk menetapkan ulang nilai awal akumulator (*accumulator* = bilangan). Operator aritmatika (+, -, *, /, %) dan operator *bitwise* (&, |) akan mengoperasikan nilai accumulator dengan bilangan input dan memperbarui nilai accumulator. Untuk operasi pembagian (/) dan modulo (%), program menyertakan validasi agar bilangan tidak bernilai 0 demi menghindari error *division/modulo by zero*. Jika pengguna memasukkan operator 'e'/'E', program mencetak pesan penutup dan langsung menghentikan eksekusi dengan *return 0*. Jika dimasukkan operator yang tidak dikenali, program mencetak pesan kesalahan dan melanjutkan perulangan.

Kode Program

```
1  #include <stdio.h>
2
3  int main() {
4      int bilangan;
5      char operator;
6      int accumulator = 0;
7      printf("Mulai perhitungan\n");
8      while (1) {
9          if (scanf("%d %c", &bilangan, &operator) != 2) {
10             break;
11         }
12         switch (operator) {
13             case 's':
14             case 'S':
15                 accumulator = bilangan;
16                 printf("= %d\n", accumulator);
17                 break;
18             case '+':
19                 accumulator += bilangan;
20                 printf("= %d\n", accumulator);
21                 break;
22             case '-':
23                 accumulator -= bilangan;
24                 printf("= %d\n", accumulator);
25                 break;
26             case '*':
27                 accumulator *= bilangan;
28                 printf("= %d\n", accumulator);
29                 break;
30             case '/':
31                 if (bilangan == 0) {
32                     printf("Error: Pembagian dengan nol tidak diperbolehkan!\n");
33                 } else {
34                     accumulator /= bilangan;
35                     printf("= %d\n", accumulator);
36                 } break;
37             case '%':
38                 if (bilangan == 0) {
39                     printf("Error: Modulo dengan nol tidak diperbolehkan!\n");
40                 } else {
41                     accumulator %= bilangan;
42                     printf("= %d\n", accumulator);
43                 } break;
44             case '&':
45                 accumulator &= bilangan;
46                 printf("= %d\n", accumulator);
47                 break;
48             case '|':
49                 accumulator |= bilangan;
50                 printf("= %d\n", accumulator);
51                 break;
52             case 'e':
53             case 'E':
54                 printf("Akhir perhitungan\n");
55                 return 0;
56             default:
57                 printf("Error: Operator tidak valid!\n");
58                 break;
59         }
60     }
61 }
```

Hasil Running

```
Mulai perhitungan
90 s
= 90
100 +
= 190
3 /
= 63
```

Pengujian Data

No	Input	Hasil Yang Diharapkan	Hasil Aktual	Keterangan
1	10 s 5 +	15	15	Valid
2	10 s 0 /	Error	Error: Pembagian dengan nol tidak diperbolehkan!	Valid
3	10 s 5 x	Error	Error: Operator tidak valid!	Valid

Analisis Hasil

Program berhasil mengimplementasikan kalkulator berbasis akumulator secara berkelanjutan. Struktur `while (1)` yang dipadukan dengan `switch-case` memungkinkan pengguna melakukan kalkulasi bertahap tanpa harus menjalankan ulang program dari awal. Adanya pengecekan `bilangan == 0` pada kasus `/` dan `%` secara efektif mencegah kesalahan *runtime*, dan penanganan operator `'e'/'E'` memberikan sarana keluar dari perulangan (*exit condition*) secara aman dan terkontrol.

4.7.4 Tahun Kabisat

Analisis Permasalahan

Tahun kabisat adalah tahun yang habis dibagi 4, tetapi untuk tahun abad (kelipatan 100), tahun tersebut harus habis dibagi 400. Logikanya: Sebuah tahun adalah kabisat jika (habis dibagi 4 DAN tidak habis dibagi 100) ATAU (habis dibagi 400).

Algoritma/ Logika Program

Program dimulai dengan membaca input sebuah tahun bernilai bulat dari pengguna yang disimpan ke dalam variabel `tahun`. Logika penentuan tahun kabisat dievaluasi menggunakan ekspresi kondisi majemuk dengan gabungan operator logika `&&` (AND) dan `||` (OR). Sebuah tahun dinyatakan sebagai tahun kabisat jika habis dibagi 4 namun tidak habis dibagi 100, ATAU habis dibagi 400. Jika ekspresi logika tersebut bernilai *true*, program mencetak informasi bahwa tahun tersebut adalah tahun kabisat. Jika bernilai *false*, program mencetak bahwa tahun tersebut bukan tahun kabisat, lalu proses selesai.

Kode Program

```
1  #include <stdio.h>
2
3  int main() {
4      int tahun;
5
6      printf("Masukkan tahun (1900-2400): ");
7      if (scanf("%d", &tahun) != 1) {
8          printf("Input tidak valid!\n");
9          return 1;
10     }
11
12     if (tahun < 1900 || tahun > 2400) {
13         printf("Tahun diluar jangkauan! Masukkan tahun antara 1900 hingga 2400.\n");
14         return 1;
15     }
16
17     if ((tahun % 400 == 0) || (tahun % 4 == 0 && tahun % 100 != 0)) {
18         printf("Tahun %d adalah Tahun Kabisat.\n", tahun);
19     } else {
20         printf("Tahun %d BUKAN Tahun Kabisat.\n", tahun);
21     }
22
23     return 0;
24 }
```

Hasil Running

```
Masukkan tahun (1900-2400): 1922
Tahun 1922 BUKAN Tahun Kabisat.
```

Pengujian Data

No	Input	Hasil yang diharapkan	Hasil aktual	Keterangan
1	1922	Bukan tahun kabisat	Tahun 1922 BUKAN Tahun Kabisat	Valid
2	2000	Tahun Kabisat	Tahun 2000 adalah Tahun Kabisat.	Valid
3	2026	Bukan Tahun kabisat	Tahun 2026 BUKAN Tahun Kabisat.	Valid

Analisis Hasil

Penggabungan operator logika && dan || mengevaluasi aturan tahun kabisat Gregorian secara tepat, membedakan tahun abad seperti 1900 (bukan kabisat) dan 2000 (kabisat).

4.7.5 Kategori Nilai

Analisis Permasalahan

Mengonversi angka nilai akhir (skala 0–100) menjadi skala huruf mutu berdasarkan standar konversi nilai akademik

Algoritma/Logika Program

Program diawali dengan meminta masukan nilai angka bertipe pecahan (*float*) dari pengguna. Sebelum mengategorikan, program melakukan validasi input menggunakan kondisi if: jika nilai kurang dari 0 atau lebih dari 100, program menampilkan pesan bahwa input tidak valid dan menghentikan proses. Jika nilai valid, program mengevaluasi rentang nilai secara bertingkat dari batas tertinggi ke terendah menggunakan struktur else-if. Nilai ≥ 85 dikategorikan sebagai 'A', nilai ≥ 70 sebagai 'B', nilai ≥ 55 sebagai 'C', nilai ≥ 40 sebagai 'D', dan nilai di bawah 40 dikategorikan sebagai 'E'. Setelah predikat huruf mutu ditentukan, program menampilkan hasil tersebut ke layar.

Kode Program

```
1  #include <stdio.h>
2  int main ()
3  {
4      int nilai;
5      printf("\n--- Cek Kelulusan Nilai ---\n");
6      printf("Masukkan nilai ujian (0-100): ");
7      scanf("%d", &nilai);
8      if (nilai >= 85 && nilai <= 100) {
9          printf("Predikat: A (Lulus dengan Sangat Baik)\n");
10     } else if (nilai >= 70 && nilai < 85) {
11         printf("Predikat: B (Lulus Baik)\n");
12     } else if (nilai >= 55 && nilai < 70) {
13         printf("Predikat: C (Cukup)\n");
14     } else if (nilai >= 0 && nilai < 55) {
15         printf("Predikat: D (Tidak Lulus)\n");
16     } else {
17         printf("Error: Nilai di luar jangkauan 0-100!\n");
18     }
19 }
```

Hasil Running

```
Masukkan nilai ujian (0-100): 90
Predikat: A (Lulus dengan Sangat Baik)
```

Pengujian Data

No	Input	Hasil yang diharapkan	Hasil Aktual	Keterangan
1	90	A	Predikat: A (Lulus dengan sangat baik)	Valid
2	60	C	Predikat: C (Cukup)	Valid

3	102	Diluar jangkauan	Error: Nilai di luar jangkauan 0-100!	Valid
---	-----	------------------	---------------------------------------	-------

Analisis Hasil

Penggunaan struktur else-if berurutan dari nilai tertinggi ke terendah menyederhanakan kondisi batas, sehingga cukup memeriksa nilai ≥ 70 setelah pengecekan nilai ≥ 85 gagal.

4.7.6 Akar Persamaan Kuadrat

Buat program untuk menghitung determinan $D = B^2 - 4AC$ dan menentukan jenis akar persamaan $Ax^2 + Bx + C = 0$ berdasarkan nilai D . Tangani juga kondisi $A = 0$.

4.7.6 Ketentuan Tugas Akar Persamaan Kuadrat

Analisis Permasalahan

- Jika $D > 0$, terdapat dua akar real berbeda.
- Jika $D = 0$, terdapat satu akar real kembar.
- Jika $D < 0$, terdapat dua akar kompleks.
- Jika $A = 0$, persamaan bukan persamaan kuadrat dan program harus memberikan pesan yang sesuai.

Algoritma/Logika Program

Program diawali dengan membaca tiga koefisien (a , b , dan c) berbentuk bilangan presisi ganda (*double*) dari pengguna. Program pertama-tama memvalidasi nilai a ; jika a bernilai 0, program memberitahukan bahwa persamaan tersebut bukan persamaan kuadrat dan langsung berhenti. Jika valid, program menghitung nilai diskriminan $D = b^2 - 4ac$. Selanjutnya, nilai D dievaluasi: jika $D > 0$, program menghitung dua akar real berbeda (x_1 dan x_2) menggunakan rumus kuadratis; jika $D = 0$, program menghitung satu akar real kembar; dan jika $D < 0$, program memisahkan perhitungan bagian real dan bagian imajiner menggunakan fungsi $\text{sqrt}(-D)$ untuk menghasilkan pasangan akar kompleks konjugat. Hasil kalkulasi akar kemudian ditampilkan lengkap sesuai bentuk kasusnya masing-masing.

Kode Program

```
1  #include <stdio.h>
2  #include <math.h>
3
4  int main() {
5      float a, b, c;
6      float d, x1, x2, riil, imajiner;
7      printf("Masukkan koefisien a, b, dan c: ");
8      if (scanf("%f %f %f", &a, &b, &c) != 3) {
9          printf("Input tidak valid!\n");
10         return 1;
11     }
12     if (a == 0) {
13         printf("Nilai 'a' tidak boleh nol pada persamaan kuadrat!\n");
14         return 1;
15     }
16     d = (b * b) - (4 * a * c);
17     printf("Nilai Determinan (D) = %.2f\n", d);
18     if (d > 0) {
19         x1 = (-b + sqrt(d)) / (2 * a);
20         x2 = (-b - sqrt(d)) / (2 * a);
21         printf("Akar-akarnya berbeda:\n");
22         printf("x1 = %.2f\n", x1);
23         printf("x2 = %.2f\n", x2);
24     }
25     else if (d == 0) {
26         x1 = -b / (2 * a);
27         printf("Akar-akarnya kembar:\n");
28         printf("x1 = x2 = %.2f\n", x1);
29     }
30     else {
31         riil = -b / (2 * a);
32         imajiner = sqrt(-d) / (2 * a);
33         printf("Akar-akarnya imajiner berlainan:\n");
34         printf("x1 = %.2f + %.2fj\n", riil, imajiner);
35         printf("x2 = %.2f - %.2fj\n", riil, imajiner);
36     }
37     return 0;
38 }
```

Hasil Running

```
Masukkan koefisien a, b, dan c: 2 3 4
Nilai Determinan (D) = -23.00
Akar-akarnya imajiner berlainan:
x1 = -0.75 + 1.20j
x2 = -0.75 - 1.20j
```

Pegujian Data

No	Input	Hasil yang diinginkan	Hasil Aktual	Keterangan
1	2 3 4	D = -23	x1 = -0.75 + 1.20j x2 = -0.75 - 1.20j	Valid
2	1 -5 6	D = 1.00	Akar-akarnya berbeda: x1 = 3.00 x2 = 2.00	Valid

3	1 2 5	$D = -16.00$	Akar-akarnya imajiner berlainan: $x_1 = -1.00 + 2.00j$ $x_2 = -1.00 - 2.00j$	Valid
---	-------	--------------	---	-------

Analisis Hasil

Penggunaan library `` dengan fungsi `sqrt()` memfasilitasi kalkulasi akar kuadrat. Pemisahan bagian real dan imajiner saat $D < 0$ memungkinkan program menangani seluruh domain kasus solusi persamaan kuadrat secara presisi.

4.8 Format Laporan Praktikum

Laporan praktikum disusun dengan urutan berikut:

- Halaman judul.
- Tujuan praktikum.
- Dasar teori singkat.
- Kode program setiap percobaan.
- Hasil running program/screenshot.
- Tabel pengujian dan hasil aktual.
- Analisis hasil percobaan.
- Jawaban pertanyaan analisis.
- Kode dan hasil tugas praktikum.
- Kesimpulan.

Pada bagian analisis, mahasiswa tidak cukup hanya menuliskan hasil running. Jelaskan hubungan antara input, kondisi, proses percabangan, dan output yang dihasilkan.

4.9 Kesimpulan

- Penggunaan `if`, `if-else`, `else-if`, dan `switch-case` memungkinkan program mengarahkan alur eksekusi logika secara fleksibel dan efisien sesuai dengan jenis kondisi yang diuji
- Penerapan penanganan kondisi khusus—seperti pencegahan pembagian atau modulo dengan nol serta pembatasan rentang input—sangat krusial untuk mencegah terjadinya kesalahan *runtime*
- Penggabungan struktur perulangan (seperti `while`) dengan operasi akumulatif dan manipulasi berbagai tipe data (integer, float, karakter, hingga operasi bitwise) membuktikan bahwa logika C dapat menangani kalkulasi kompleks yang berkelanjutan secara interaktif dan presisi.

4.10 Checklist Praktikum

- ☒ Semua program berhasil dikompilasi.
- ☒ Setiap program telah diuji dengan beberapa data.
- ☒ Hasil running telah didokumentasikan.
- ☒ Analisis setiap percobaan telah dikerjakan.
- ☒ Semua tugas telah dikerjakan.
- ☒ Kode program telah diberi indentasi dan komentar seperlunya.